Team Task Manager - Technical Overview & Code Review

1. Project Overview & Architecture

The **Team Task Manager** is a full-stack web application designed for managing collaborative projects. It enables role-based access control where Admins can manage projects and tasks, while Members can track their assignments and update their progress.

Tech Stack (MERN + Vite)

- **MongoDB & Mongoose**: A NoSQL database that offers flexible schema structures. Used to model `Users`, `Projects`, and `Tasks` with built-in reference (`ref`) capabilities for relational ties.
- **Express.js & Node.js**: The backend framework. Provides a fast, minimalist routing layer to expose RESTful APIs.
- **React.js & Vite**: The frontend framework, bundled with Vite for ultra-fast Hot Module Replacement (HMR). Uses Context API for global state.
- **Vanilla CSS**: Used for styling the application with a highly customizable, modern, glassmorphic dark-theme aesthetic, without relying on utility classes like Tailwind.

System Flow

1. **Authentication**: Users sign up/log in to receive a JSON Web Token (JWT).
2. **Authorization Interceptor**: The frontend uses an Axios interceptor to automatically attach the JWT as a `Bearer` token to all subsequent API requests.
3. **Role-based Rendering**: The UI dynamically alters its layout based on the user's role (e.g., hiding the "New Project" button for Members).
4. **Backend Validation**: The Express middleware verifies the JWT and ensures the user has the required `Admin` privileges before executing restricted CRUD operations.

---

2. Component Structure & Data Flow

Backend (`/server`)

- **Models**: Defines the MongoDB schemas (`User.js`, `Project.js`, `Task.js`). Notice how `Project` keeps an array of `members`, and `Task` references `assignee`.
- **Controllers**: Contains the core business logic. E.g., `dashboardController.js` aggressively filters tasks depending on whether the requesting user is an Admin (sees all tasks in their projects) or a Member (sees only assigned tasks).
- **Middleware**: `auth.js` intercepts requests to validate the JWT and fetch the user context.

Frontend (`/client`)

- **AuthContext (`AuthContext.jsx`)**: Centralizes the authentication state. Exposes `user`, `login`, `register`, and `logout` functions globally.
- **Pages**:
  - `Dashboard.jsx`: Fetches and displays aggregate statistics (overdue tasks, completion rates).
  - `Projects.jsx`: Fetches and maps over user-accessible projects.
  - `Board.jsx`: The core interactive component. It fetches tasks for a specific project, organizes them by status, and implements HTML5 drag-and-drop.

- **Components (`TaskModal.jsx` , `ProjectModal.jsx`)**: Reusable forms utilizing React state for controlled inputs, allowing for the creation and editing of resources.

---

3. Key Features Explained

Drag-and-Drop Kanban Board

The board uses the native HTML5 Drag and Drop API.

- `onDragStart` : Attaches the `taskId` to the `dataTransfer` object.
- `onDragOver` : Calls `e.preventDefault()` to allow the drop event.
- `onDrop` : Reads the `taskId` , verifies if the user has permission to move the task, and executes a `PUT` request to update the task's status.

Role-Based Access Control (RBAC)

Implemented on both sides:

- **Frontend**: Conditionally renders elements (`{user?.role === 'Admin' && <button>Add Task</button>}`).
- **Backend**: Employs the `adminOnly` middleware on critical routes, ensuring that even if a user bypasses the UI, the API rejects the request with a `403 Forbidden` .

---

4. Code Review & Areas for Improvement

While the application is fully functional, robust, and cleanly structured, here is an honest technical code review and potential areas for improvement.

Pros

- **Clean Architecture**: Strong separation of concerns. Controllers, routes, and models are clearly decoupled.
- **Security**: Passwords are securely hashed using `bcryptjs` . API endpoints are protected using JWTs.
- **Modern UI/UX**: The application uses pure CSS for a lightweight but highly premium glassmorphism effect.
- **Responsive State**: The use of React Context prevents prop-drilling for authentication data.

Cons & Technical Debt

- **Error Handling**: Currently, the frontend relies on `err.response?.data?.message || 'Error'` . This can be fragile. Implementing a centralized toast notification system would improve the user experience.
- **Data Fetching Strategy**: Data is currently fetched using `useEffect` and standard `useState` . While fine for small apps, as the app grows, using a library like **React Query** or **SWR** would provide better caching, optimistic UI updates, and reduce duplicate requests.
- **Drag-and-Drop Visuals**: The native HTML5 drag-and-drop is functional but lacks smooth animations during the drag state. Libraries like `@hello-pangea/dnd` would make the interactions feel smoother.

Future Improvements

1. **WebSocket Integration**: Implement `Socket.io` to allow real-time updates when another team member moves a task on the board.

2. **Project Members Management UI**: Add a dedicated screen for Admins to search and add existing users to their projects dynamically.
3. **Pagination & Filtering**: The `getTasksByProject` query will eventually become slow if a project has thousands of tasks. Pagination and index optimization on the `project` field in MongoDB would be necessary.

---

5. Website Flow & UI Walkthrough

Admin Workflow

- **Login/Register**: Admins sign up by selecting the "Admin" role from the dropdown.
- **Sidebar Navigation**: Grants access to the **Dashboard** and **Projects** views.
- **Dashboard**: The admin sees aggregate statistics for *all* projects they own, including Total Projects, Active Tasks, and Overdue Tasks.
- **Projects Page**: The "New Project" button is visible exclusively to admins. Clicking it opens the `ProjectModal` to create a new workspace. Clicking an existing Project Card routes the admin to the Board.
- **Kanban Board (`/projects/:id`)**: The "Add Task" button is visible. Clicking it opens the `TaskModal`, allowing the admin to input a Title, Description, Priority, and assign it to a specific user. Admins have full access to drag-and-drop any task across the columns to update statuses.

Member (User) Workflow

- **Login/Register**: Users sign up by selecting the "Member" role.
- **Dashboard**: The statistics filter specifically to tasks where the member is assigned.
- **Projects Page**: The "New Project" button is completely hidden. The user only sees projects they have been invited to by an Admin.
- **Kanban Board (`/projects/:id`)**: The "Add Task" button is hidden. Members can see all tasks, but they are only authorized to drag-and-drop tasks that are explicitly assigned to them. If they click their assigned task, a simplified `TaskModal` appears, restricting them to only updating the task's "Status" (they cannot edit the title or description).

---

6. Presentation Script (5-Minute Video Walkthrough)

Use this script while sharing your screen in VS Code to explain your project to the reviewer.

[0:00 - 0:45] Introduction & Project Goal

> *"Hi everyone, today I'll be walking you through my Team Task Manager. This is a full-stack MERN application designed to help teams collaborate using a Kanban-style board. My main focus was on implementing strict Role-Based Access Control, separating 'Admins' from 'Members', and creating a premium glassmorphic UI using pure CSS without relying on external UI libraries."*

[0:45 - 1:45] Backend Architecture & Database Connection *(Open `server/index.js` and `server/.env` in VS Code)*

> *"Let's start with the backend. I used Express and Node.js. Here in `index.js`, you can see the main entry point where I configure CORS and load my environment variables. To attach the database, I used Mongoose. I store my MongoDB connection string in a `.env` file, and call `mongoose.connect()`. If we look inside the `server/models` folder, you'll see my three main schemas: `User`, `Project`, and `Task`. A key design decision here is the use of Mongoose References. For example, a `Task`*

> *references a `Project` ID and an `Assignee` ID, allowing me to build relational queries within a NoSQL database."*

[1:45 - 2:45] Authentication & Security Logic *(Open `server/middleware/auth.js` and `server/controllers/authController.js`)*

> *"For security, I implemented JWT-based authentication. In the `authController`, when a user registers, their password is encrypted using `bcryptjs` before hitting the database. The core security logic lives in `middleware/auth.js`. I have two middlewares: `protect`, which extracts and verifies the JWT token from the Authorization header, and `adminOnly`, which checks the decoded token to ensure the user has the 'Admin' role before they can hit sensitive endpoints like creating projects."*

[2:45 - 3:45] Frontend Architecture & Global State *(Open `client/src/context/AuthContext.jsx` and `client/src/services/api.js`)*

> *"Moving to the frontend, the app is built with React and Vite. To manage global state without prop-drilling, I used the React Context API. Inside `AuthContext.jsx`, I manage the user's session and handle login/logout logic. I also created a centralized Axios instance in `api.js`. This is crucial because it uses an interceptor to automatically attach the user's JWT token to the header of every outgoing request, saving me from having to manually pass it in every component."*

[3:45 - 4:45] UI Implementation & Drag-and-Drop *(Open `client/src/pages/Board.jsx` and `client/src/index.css`)*

> *"Finally, let's look at the UI. The Kanban board in `Board.jsx` maps over the tasks and groups them by status. I implemented the drag-and-drop functionality using the native HTML5 Drag and Drop API—capturing the task ID on `onDragStart` and firing a PUT request to the backend on `onDrop`. For the styling, I completely customized `index.css` using CSS variables and backdrop-filters to achieve a sleek glassmorphism effect, giving the app a very modern, premium feel."*

[4:45 - 5:00] Conclusion

> *"To wrap up, this project demonstrates a strong understanding of full-stack data flow, security paradigms, and modern React development. Thank you for your time, and I'd be happy to answer any questions."*